

12. Bidding and Market Participation

The Missing Link: From Schedule to Offer

Chapters 5 through 10 built a complete dispatch pipeline: forecast prices, solve an LP, execute the first period's charge or discharge decision, observe the actual price, and repeat. But every chapter so far has quietly dodged a critical reality: **the battery does not choose how much to charge or discharge**. It submits offers to AEMO, and AEMO's dispatch engine decides what the battery actually does.

The distinction matters enormously. In the LP world of Chapter 9, you solve for the optimal energy quantity at each time step, and you implicitly assume the market will let you trade that quantity at the forecast price. In the real NEM, you submit price-quantity bands *before* you know the clearing price, and you are dispatched only if the price clears past your band. The dispatch engine -- NEMDE -- sees your offers alongside every other generator and load in the market, stacks them, and clears them simultaneously. Your battery is one participant in a multi-billion-dollar auction that runs every five minutes, 288 times per day, 365 days per year.

NEMDE (NEM Dispatch Engine): AEMO's central dispatch algorithm that clears the energy and FCAS markets simultaneously every five minutes. NEMDE takes all generator offers and load bids, applies network constraints, loss factors, and inter-regional transfer limits, and finds the least-cost dispatch that meets demand. The clearing price in each region is the cost of the marginal unit of energy -- the most expensive generator dispatched to meet the last increment of demand.

This chapter bridges the gap between the mathematical optimum (the LP solution from Chapter 9) and the physical market mechanism (submitting offers to NEMDE). It introduces two simulation modes: **price-taker mode**, where the battery's actions do not affect the clearing price (what we have done so far), and **price-maker mode**, where we insert the battery's offers into the real bid stack using nempy and re-clear the market, allowing the battery's own bids to move the price.

The core question of this chapter: How do you translate "I want to discharge 100 MW at 6 PM" into a set of price-quantity offer bands that cause NEMDE to dispatch you at 100 MW when conditions are right -- and cause NEMDE to not dispatch you when conditions are wrong? And when does the act of offering 100 MW itself change the price you receive?

The NEM Offer Structure

Price-Quantity Bands

Every generator and load in the NEM submits offers structured as up to **ten price-quantity bands**. Each band specifies a price (in \$/MWh) and a quantity (in MW). The bands must be in ascending price order and together must cover the unit's full registered capacity.

Offer band: A single price-quantity pair within a generator's offer. Each band says: "I am willing to supply this many MW at or above this price." A generator may submit up to 10 bands, covering its full registered capacity. NEMDE dispatches the cheapest bands first, working up the price stack until demand is met.

For a 100 MW battery like the Mannum BESS, the ten bands might look like this for a discharge offer:

Band	Price (\$/MWh)	Quantity (MW)	Intent
1	-\$1,000	0	Floor band (empty -- not willing to discharge at negative prices)
2	\$50	10	Small tranche: discharge if price exceeds \$50
3	\$80	15	Additional capacity above \$80
4	\$120	20	Core discharge range
5	\$150	20	Core discharge range
6	\$200	15	Higher-value tranche
7	\$300	10	Reserved for moderate spikes
8	\$500	5	Reserved for significant spikes
9	\$1,000	3	Reserved for severe spikes
10	\$16,600	2	Market price cap -- last resort

The NEM's price range is from the market floor price of -\$1,000/MWh to the market price cap (MPC) of \$16,600/MWh (as of 2024-25). Any band price must fall within this range.

Market price cap (MPC): The maximum price allowed in the NEM spot market, set by the AER and adjusted annually for inflation. As of 2024-25, the MPC is \$16,600/MWh. No generator can offer above this price. The MPC exists to limit the cost of extreme price events to consumers while still providing an incentive for peaking generation to be available.

Market floor price (MFP): The minimum price allowed in the NEM, currently -\$1,000/MWh. Negative prices occur when generators -- typically wind and solar with renewable energy certificate revenue -- are willing to pay to keep running rather than shut down. Batteries benefit from negative prices: they earn revenue by charging (consuming energy) when the price is negative.

How Dispatch Works

When NEMDE runs (every five minutes), it collects all offers, builds the aggregate supply curve, and dispatches to meet demand:

1. **Stack all bands** from all generators in ascending price order
2. **Dispatch from the bottom** -- cheapest bands first
3. **Stop when supply meets demand** -- the price of the last dispatched band sets the regional clearing price
4. **All dispatched generators receive the clearing price**, regardless of their offer price

Example -- battery dispatch at the margin: Suppose the SA1 clearing price settles at \$130/MWh. The Mannum BESS offer above has bands 1-4 fully dispatched ($0 + 10 + 15 + 20 = 45$ MW), and band 5 (priced at \$150) is not dispatched because $\$150 > \130 . The battery discharges 45 MW and receives \$130/MWh for every MWh it delivers -- not the band prices of \$50, \$80, or \$120. This is the ****uniform pricing**** mechanism: the clearing price applies to all dispatched energy, regardless of offer price.

Uniform pricing: A market mechanism where all dispatched generators receive the same price -- the marginal clearing price -- regardless of their individual offer prices. This is sometimes called "pay-as-cleared" pricing. It gives generators an incentive to offer at their true marginal cost (zero for batteries, which have no fuel cost) rather than inflate their offers, because offering higher than necessary risks not being dispatched at all.

The Bidirectional Battery

A battery is unique in the NEM because it participates on both sides of the market. It acts as a **generator** when discharging (supplying energy to the grid) and as a **load** when charging (consuming energy from the grid).

Integrated resource: A resource that can both generate and consume electricity, participating on both sides of the energy market. In the NEM, batteries were historically required to register as both a "Generator" and a "Load" with separate dispatch unit identifiers (DUIDs). Under the NEM reform program, batteries can now register as a single Integrated Resource System (IRS), with a single DUID and coordinated dispatch across charge and discharge. The Mannum BESS operates under this framework.

Under the Integrated Resource System (IRS) framework, the Mannum BESS submits:

- **Discharge offers:** up to 10 price-quantity bands for generation, priced from low to high. Dispatched when the clearing price exceeds the band price.
- **Charge bids:** up to 10 price-quantity bands for load, priced from high to low. Dispatched when the clearing price falls below the band price.

The symmetry is important: discharge offers go *up* the price stack (the battery wants to sell at high prices), while charge bids go *down* the price stack (the battery wants to buy at low prices).

Example -- charge bids for the Mannum BESS: | Band | Price (\$/MWh) | Quantity (MW) | Intent | |-----|-----|-----|-----| | 1 | \$16,600 | 0 | Ceiling band (empty -- not willing to charge at extreme prices) | | 2 | \$80 | 5 | Small charge if price drops below \$80 | | 3 | \$50 | 15 | Moderate charge below \$50 | | 4 | \$30 | 20 | Bulk charging range | | 5 | \$10 | 20 | Bulk charging range | | 6 | \$0 | 15 | Charge at zero or negative prices | | 7 | -\$20 | 10 | Charge during mild negative prices | | 8 | -\$50 | 5 | Charge during moderate negative prices | | 9 | -\$200 | 5 | Charge during significant negative prices | | 10 | -\$1,000 | 5 | Market floor -- absorb in extreme negative events | If the clearing price is \$25/MWh, bands 4-10 are dispatched ($20 + 20 + 15 + 10 + 5 + 5 + 5 = 80$ MW of charging). The battery pays \$25/MWh for every MWh it consumes -- again, uniform pricing.

The offer bands encode the dispatch strategy. A battery's ten bands are not arbitrary -- they represent its view of the price distribution. Aggressive discharge bands (low prices, high quantities) mean the battery expects to discharge frequently. Conservative discharge bands (high prices, low quantities) mean the battery is waiting for spikes. The optimal band placement depends on the probabilistic forecast from Chapter 8 and the scenario analysis from Chapter 9.

Rebidding: Updating Offers in Real Time

The Rebidding Mechanism

The NEM allows generators to update their offers at any time before gate closure. This is called **rebidding**, and it is one of the most distinctive features of the NEM compared to other electricity markets.

Rebidding: The process of updating a generator's price-quantity offer bands after the initial submission but before gate closure. Rebids can change the quantities allocated to each band (shifting capacity between price levels) but must maintain the same ten band prices within a trading day. Rebids take effect from the next dispatch interval after AEMO processes them.

Gate closure: The deadline after which no further rebids are accepted for a given dispatch interval. In the NEM, gate closure is approximately 5 minutes before the start of the dispatch interval -- effectively, the most recent rebid received before NEMDE runs for that interval is the one used. This near-real-time gate closure is unusually short by international standards (many European markets close hours ahead) and gives NEM participants enormous flexibility to respond to changing conditions.

The key constraint on rebidding: within a single trading day (4:00 AM to 4:00 AM AEST), the ten band **prices** are fixed. When you rebid, you can only change the **quantities** allocated to each band. To change your band prices, you must wait for the next trading day -- or submit a new set of band prices, which resets all your bands for the remainder of the day.

Example -- rebidding in response to a forecast update: At 2:00 PM, the Mannum BESS has 80% state of charge and its original offer has 20 MW at \$120 and 20 MW at \$150 for the 5:00 PM interval. At 3:30 PM, the updated weather forecast shows cloud cover clearing earlier than expected, meaning solar generation will persist through sunset. The trader rebids: move the 20 MW from band 4 (\$120) up to band 7 (\$300), reasoning that the evening price spike will be delayed and muted by the extra solar. The rebid reason logged with AEMO: "Updated solar generation forecast -- increased PV output expected through 5:30 PM." If the price does spike to \$250 at 5:00 PM, the rebid means only $10 + 15 = 25$ MW of discharge is dispatched (bands 2-3), compared to 45 MW without the rebid. The remaining energy is preserved for later, higher-priced intervals.

Rebid Reasons and Compliance

Every rebid must include a **reason** that explains the change. AEMO and the AER review rebid reasons to detect market manipulation -- rebids that are designed to inflate prices rather than respond to genuine changes in conditions.

Acceptable rebid reasons include:

- Changes in demand or price forecasts
- Changes in weather forecasts (wind, solar, temperature)
- Changes in plant availability or equipment status
- Changes in fuel supply or cost
- Changes in interconnector flows or network constraints
- Changes in the unit's own state of charge (for batteries)
- Commercial decisions based on updated market information

The rebidding obligation: The NEM's rebidding rules require that the stated reason must be genuine -- the trader must actually be responding to the stated factor. "Rebidding to maximise profit" is not a valid reason on its own; the trader must point to a specific change in conditions (even if the effect of the change is to increase expected profit). The AER actively monitors rebidding behaviour and has taken enforcement action against generators whose rebidding patterns suggest strategic manipulation rather than genuine responses to changing conditions.

Rebidding Strategy for Batteries

Batteries rebid more frequently than most other NEM participants because their state of charge changes throughout the day, and their optimal strategy depends on remaining energy inventory. A battery that has already discharged half its energy has a fundamentally different optimal offer than one at full charge.

Common rebidding triggers for the Mannum BESS:

Trigger	Response
SoC drops below 30%	Withdraw to higher bands
Price spike in progress	Shift to lower bands, dispatch now
Solar forecast revised up	Shift charge lower; delay discharge
Interconnector binding	Increase offer if constrained
FCAS prices rising	Reposition capacity to FCAS

The Emulator: Two Modes of Simulation

Why Simulate the Market?

Throughout Chapters 5-10, we evaluated dispatch strategies by a simple method: solve the LP for the optimal schedule, then compute revenue by multiplying the dispatch quantities by the historical prices. This is **settlement on historical prices** -- the standard backtest.

But this approach embeds a hidden assumption: **the battery's dispatch does not change the market price**. When we solve the LP and dispatch 100 MW of discharge at 6 PM, we assume the price at 6 PM is the same whether the battery discharges or not. For a small battery in a large market, this is a reasonable approximation. For a 100 MW battery in SA1 -- a region where total demand might be 1,200-2,000 MW -- the battery's dispatch can shift the clearing price by several dollars per MWh or more.

This section introduces two simulation modes that handle this assumption differently.

Price-Taker Mode

Price-taker: A market participant whose actions are too small to affect the clearing price. A price-taker faces an exogenous price -- it can choose how much to buy or sell, but the price it receives is determined by other market forces. In the NEM, small generators (a few MW) are effectively price-takers. Whether a 100 MW battery is a price-taker depends on the region's size and the shape of the supply stack.

Price-taker mode is what the course has used so far:

1. **Forecast** prices for the next 48 half-hours using the models from Chapters 6-8
2. **Optimise** the dispatch schedule using the LP from Chapter 9 (scenario MPC or chance-constrained MPC)
3. **Settle** on historical prices -- multiply the dispatch quantities by the actual prices that occurred
4. **Compute** revenue and capture ratio

The price-taker assumption makes backtesting simple and fast. You do not need to model the bid stack, run NEMDE, or account for market impact. The LP from Chapter 9 is the only optimisation required.

```
# Price-taker backtest (simplified from Chapter 9) def backtest_price_taker(forecasts, actuals, battery): """Run rolling-origin backtest in price-taker mode. Args: forecasts: Quantile forecasts, shape (n_origins, horizon, n_quantiles). actuals: Actual prices, shape (n_periods,). battery: Battery parameters dict (power_mw, duration_h, rte). Returns: DataFrame with columns: dispatch_mw, price, revenue. """ results = [] for origin in range(len(forecasts)): # Solve the LP using forecast prices schedule = solve_dispatch_lp(forecasts[origin], battery, method="chance_constrained", quantile_pair=(0.10, 0.90), ) # Settle on actual prices (price-taker assumption) revenue = schedule["dispatch_mw"] * actuals[origin : origin + horizon] results.append(revenue) return pd.concat(results)
```

Price-Maker Mode

Price-maker: A market participant large enough that its actions measurably affect the clearing price. A price-maker faces a downward-sloping residual demand curve: the more it supplies, the lower the clearing price it receives. In the NEM, large generators (hundreds of MW) and large batteries in small regions are price-makers. The 100 MW Mannum BESS is at the boundary -- its market impact is non-negligible in SA1, especially during low-demand periods.

Price-maker mode uses **nempy** -- the open-source NEM dispatch engine developed by UNSW-CEEM -- to simulate the actual clearing process with the battery's offers inserted into the real bid stack.

nempy: An open-source Python implementation of the NEM dispatch engine (NEMDE), developed by the Collaborative Centre for Energy and Environmental Markets (CEEM) at UNSW. nempy takes historical bid stacks (from NEMOSIS), network constraints, interconnector models, and FCAS requirements, and re-clears the market to produce dispatch targets and prices. It can be used to simulate counterfactual market outcomes -- "what would the price have been if this battery had offered differently?"

The price-maker simulation workflow:

1. **Pull the historical bid stack** for the interval using NEMOSIS (the same data source from Chapter 1). This gives every generator's actual offer bands for that interval.
2. **Remove the battery** (or its predecessor, if applicable) from the bid stack to create a counterfactual "world without the battery."
3. **Insert the battery's new offer bands** -- the ten discharge bands and ten charge bands constructed from the forecast.
4. **Clear the market** using nempy, applying all network constraints, loss factors, and FCAS requirements.
5. **Read the results:** the new clearing price, the battery's dispatched quantity, and the dispatch of every other generator.

```
# Price-maker simulation using nempy (conceptual) import nempy from nempy import markets def simulate_price_maker(interval, battery_offer, battery_bid): """Simulate one dispatch interval with battery offers in the bid stack. Args: interval: Datetime of the dispatch interval. battery_offer: DataFrame with columns [band, price, quantity] for discharge. battery_bid: DataFrame with columns [band, price, quantity] for charge. Returns: dict with keys: clearing_price, dispatch_mw, total_revenue. """ # Create the nempy market instance market = markets.SpotMarket( market_regions=["SA1"], unit_info=get_unit_info(interval), # All generators ) # Load historical bid stacks from NEMOSIS volume_bids, price_bids = load_bid_stack(interval, region="SA1") # Remove the battery's existing bids (if present) volume_bids = volume_bids[volume_bids["DUID"] != "MNNMBESS1"] price_bids = price_bids[price_bids["DUID"] != "MNNMBESS1"] # Insert the battery's new offer bands volume_bids = pd.concat([volume_bids, battery_offer]) price_bids = pd.concat([price_bids, format_price_bands(battery_bid)]) # Set bids in the market market.set_unit_volume_bids(volume_bids) market.set_unit_price_bids(price_bids) # Load demand, interconnectors, constraints market.set_demand_constraints(get_demand(interval, "SA1")) market.set_interconnectors(get_interconnector_definitions()) # Dispatch market.dispatch() # Extract results price = market.get_energy_prices()["SA1"] dispatch = market.get_unit_dispatch() battery_dispatch = dispatch.loc[dispatch["DUID"] == "MNNMBESS1_NEW", "DISPATCH"].values[0] return { "clearing_price": price, "dispatch_mw": battery_dispatch, "total_revenue": battery_dispatch * price / 12, # 5-minute interval }
```

The key difference: In price-taker mode, the battery chooses a quantity and receives the historical price. In price-maker mode, the battery submits offer bands, nempy re-clears the market, and the battery receives a *new* price that reflects its own participation. When the battery discharges 100 MW, it adds 100 MW of supply to SA1, which pushes the clearing price down. When it charges 100 MW, it adds 100 MW of demand, which pushes the price up. Both effects reduce the battery's revenue compared to the price-taker assumption.

What nempy Does Under the Hood

nempy replicates the core logic of NEMDE:

1. **Builds the LP:** minimise total dispatch cost (equivalently, clear at the lowest possible prices) subject to:
2. Supply meets demand in each region
3. Each generator dispatched within its offered bands
4. Interconnector flows within thermal and stability limits
5. Loss factors applied to inter-regional transfers
6. FCAS requirements met (if modelled)

Ramp rate constraints (generator output cannot change faster than its ramp rate)

Solves the LP: uses a standard LP solver (HiGHS or CPLEX) to find the cost-minimising dispatch

Extracts dual variables: the shadow price on each region's demand constraint is the clearing price -- the cost of serving one more MW of demand in that region

The clearing price from nempy matches the historical AEMO price when the same bid stack and constraints are used. Inserting the battery's offers changes the supply curve and therefore changes the clearing price -- this is the market impact.

Example -- market impact of the Mannum BESS: Consider a low-demand SA1 interval where demand is 1,200 MW. Without the battery, the marginal generator is a gas plant offering at \$85/MWh. The Mannum BESS offers to discharge 100 MW in bands priced between \$50 and \$200. With the battery's offers inserted: - nempy dispatches the battery's cheap bands (say 45 MW at bands priced \$50-\$120) before reaching the gas plant - The gas plant is partially displaced -- it now dispatches 955 MW instead of 1,000 MW - But the gas plant's next band (at \$78/MWh) becomes the marginal unit, not its \$85 band - The clearing price drops from \$85 to \$78 The battery receives \$78/MWh instead of \$85/MWh -- a \$7/MWh impact. On 45 MW over a 5-minute interval, this costs the battery $\$7 \cdot 45 / 12 = \26.25 per interval, or about \$3,150 per hour of sustained dispatch. Over a year, market impact can cost a 100 MW battery hundreds of thousands of dollars in SA1.

Price-Taker vs Price-Maker: When Market Impact Matters

The Residual Demand Curve

The core concept for understanding market impact is the **residual demand curve**: the demand that remains after all other generators have been dispatched. The battery faces this residual demand -- it is competing for the marginal MW of supply.

Residual demand curve: The demand curve that a single generator faces after accounting for the supply from all other generators. It is computed by subtracting the aggregate supply curve of all other generators from the total demand curve. The slope of the residual demand curve determines how much the clearing price changes when the generator changes its output -- a steep residual demand curve means the price is insensitive to the generator's output (price-taker territory), while a flat residual demand curve means the price moves significantly (price-maker territory).

The residual demand curve can be constructed from the historical bid stack:

1. Pull all generator offers for the interval from NEMOSIS
2. Remove the battery's offers
3. Build the aggregate supply curve from the remaining offers (cumulative capacity vs price)
4. The residual demand at any price level p is: total demand minus the supply from other generators at price p

The slope of the residual demand curve at the clearing price determines the battery's **market impact factor** -- how much the price moves per MW of battery dispatch.

Figure 12.1 — The residual demand curve for SA1. The left panel shows the full supply stack with the battery's position marked. The right panel shows the residual demand curve -- the demand remaining after all other generators are dispatched. The slope of the residual demand curve at the clearing point determines the battery's market impact. During low-demand periods with a steep supply stack, even 100 MW of battery dispatch barely moves the price. During high-demand periods near the top of the stack, 100 MW can shift the price by tens of dollars per MWh.

The Market Impact Factor

Market impact factor: The change in clearing price caused by one additional MW of supply from the battery. Mathematically, it is the inverse of the slope of the residual demand curve at the operating point: $\text{impact} = -dp/dq$, where p is the clearing price and q is the battery's dispatch. Units are \$/MWh per MW. A market impact factor of \$0.05/MW means that each additional MW of battery discharge reduces the clearing price by \$0.05/MWh.

The market impact factor varies enormously depending on market conditions:

Condition	Supply stack shape	Impact factor (\$/MWh per MW)	Impact on 100 MW battery
Low demand, mid-merit	Gradual slope	\$0.01-\$0.05	\$1-\$5/MWh price shift
Moderate demand, coal-gas transition	Moderate step	\$0.05-\$0.20	\$5-\$20/MWh price shift
High demand, near capacity	Steep hockey stick	\$0.50-\$5.00	\$50-\$500/MWh price shift

Extreme low demand, renewables dominant	Nearly flat	\$0.001-\$0.01	<\$1/MWh price shift
---	-------------	----------------	----------------------

Example -- when does market impact matter for the Mannum BESS? Consider the 100 MW / 200 MWh Mannum BESS in SA1, where typical demand ranges from 800 MW (overnight) to 2,500 MW (summer afternoon peak). **At 3:00 AM, demand = 900 MW:** The supply stack is in the flat coal/wind section. The marginal generator is a coal plant with bands spread over \$25-\$40/MWh. The impact factor is about \$0.02/MW. Discharging 100 MW shifts the price by about \$2/MWh -- negligible. The price-taker assumption loses less than 3% of revenue. **At 6:00 PM, demand = 1,800 MW:** The supply stack is at the coal-gas transition. The next 200 MW of supply jumps from \$50 to \$120/MWh in discrete steps. The impact factor is about \$0.15/MW. Discharging 100 MW shifts the price by about \$15/MWh. If the historical price was \$130, the battery receives \$115 -- an 11.5% revenue loss compared to the price-taker assumption. **At 4:30 PM on a 42-degree day, demand = 2,400 MW:** The stack is near the top of the hockey stick. The next 50 MW jumps from \$200 to \$800. The impact factor is about \$2/MW. Discharging 100 MW collapses the price from \$800 to \$600 -- a 25% revenue loss. But the revenue is still enormous (\$600/MWh), and the battery is providing essential reliability to the grid.

Revenue Loss from Market Impact

The total revenue loss from market impact depends on two factors: the market impact factor and the dispatch quantity. The relationship is **quadratic** -- revenue loss grows with the square of the dispatch quantity because each additional MW both reduces the price *and* applies that lower price to all previously dispatched MW.

The revenue loss per interval is approximately:

$$\text{Revenue loss} \approx (1/2) \cdot \text{impact_factor} \cdot \text{dispatch_mw}^2 \cdot \text{interval_hours}$$

This quadratic relationship means that doubling the battery's power capacity quadruples the revenue loss from market impact. A 50 MW battery in SA1 might lose 2% of revenue to market impact, while a 200 MW battery might lose 8% -- the loss grows faster than the capacity.

Figure 12.2 — Price-taker vs price-maker revenue as a function of battery size. The left panel shows total annual revenue under each assumption. The right panel shows the revenue gap (price-taker revenue minus price-maker revenue) as a percentage. For the 100 MW Mannum BESS, the gap is approximately 4-8% of annual revenue, depending on market conditions. The gap grows quadratically with battery size, reaching 15-20% for a hypothetical 300 MW battery in SA1.

The practical threshold: As a rule of thumb, market impact becomes economically significant (costing more than 3% of revenue) when the battery's power capacity exceeds about 5% of the region's typical minimum demand. For SA1, with minimum demand around 800-1,000 MW, this threshold is about 40-50 MW. The 100 MW Mannum BESS is clearly above this threshold -- price-maker modelling is not optional for accurate revenue estimation.

The Self-Cannibalization Problem

Market impact creates a **self-cannibalization** effect: the battery's own dispatch reduces the price it receives. Discharge pushes prices down (more supply); charging pushes prices up (more demand). Both effects work against the battery.

This self-cannibalization is particularly severe for batteries because they participate on both sides of the market:

- **During discharge:** The battery adds supply, pushing the price down. The more it discharges, the lower the price it receives.
- **During charging:** The battery adds demand, pushing the price up. The more it charges, the higher the price it pays.
- **Net effect:** The arbitrage spread (discharge price minus charge price) is narrower in reality than the historical prices suggest.

Example -- self-cannibalization in both directions: Historical prices show a \$30/MWh overnight low and a \$150/MWh evening peak -- an arbitrage spread of \$120/MWh. The price-taker revenue for 100 MW · 2h = 200 MWh is $\$120 \cdot 200 / 2 = \$12,000$ (simplified, ignoring efficiency). In price-maker mode: - Charging 100 MW overnight pushes the price from \$30 to \$38 (added demand in a flat part of the stack) - Discharging 100 MW in the evening pushes the price from \$150 to \$138 (added supply in a steeper part of the stack) - The effective spread is $\$138 - \$38 = \$100/\text{MWh}$ instead of \$120/MWh - Revenue drops from \$12,000 to \$10,000 -- a 17% reduction For a smaller battery (25 MW), the price shifts would be roughly \$2 and \$3, giving a spread of \$145/MWh and a revenue reduction of only 4%.

Offer Construction from Scenarios

From Probabilistic Forecast to Offer Bands

The probabilistic forecast from Chapter 8 provides quantile predictions for each dispatch interval -- a distribution of possible prices. The copula-based scenario generation from Insert C in Chapter 8 converts these quantiles into complete, temporally correlated price trajectories. Now we need to translate these into the ten price-quantity bands required by NEMDE.

The fundamental insight: **the offer bands should reflect the battery's uncertainty about the price.** Place discharge capacity at price levels where you believe the clearing price is likely to be above your band (so you will be dispatched). Place charge capacity at price levels where you believe the clearing price is likely to be below your band (so you will be dispatched for charging).

The Quantile-to-Band Mapping

The most principled approach maps forecast quantiles directly to offer bands:

Discharge offer construction:

1. From the copula scenarios (Chapter 8), compute the quantile forecast for the current interval: $q_{0_0_5}$, $q_{0_1_0}$, $q_{0_2_0}$, $q_{0_3_0}$, $q_{0_4_0}$, $q_{0_5_0}$, $q_{0_6_0}$, $q_{0_7_0}$, $q_{0_8_0}$, $q_{0_9_0}$, $q_{0_9_5}$.
2. Assign each of the 10 offer bands a price equal to one of these quantiles (approximately).
3. Assign quantities to each band based on the MPC's optimal dispatch at that price level.

A practical algorithm:

```
def construct_discharge_offer(quantile_forecast, battery, n_bands=10): """Construct 10-band discharge offer from
quantile forecast. Args: quantile_forecast: dict mapping quantile level to price, e.g. {0.05: 20, 0.10: 28, 0.25: 45,
0.50: 85, 0.75: 140, 0.90: 250, 0.95: 400}. battery: dict with keys power_mw, duration_h, rte. n_bands: Number of
bands (NEM maximum is 10). Returns: DataFrame with columns [band, price, quantity]. """ power = battery["power_mw"] #
100 MW for Mannum # Select quantile levels for band prices # Discharge bands: price from low to high # Place most
capacity around the median and above band_quantiles = [0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.90, 0.95]
bands = [] for i, q in enumerate(band_quantiles): price = quantile_forecast[q] # Allocate more capacity to bands near
the median # (where dispatch is most likely to be partially filled) if 0.30 <= q <= 0.70: qty = power * 0.15 # 15 MW
each for core bands elif q >= 0.80: qty = power * 0.05 # 5 MW each for tail bands else: qty = power * 0.10 # 10 MW
each for lower bands bands.append({"band": i + 1, "price": price, "quantity": qty}) # Adjust quantities to sum to
total power df = pd.DataFrame(bands) df["quantity"] = df["quantity"] * power / df["quantity"].sum() return df
```

Charge bid construction follows the same logic but inverted -- place charge capacity at low quantiles:

```
def construct_charge_bid(quantile_forecast, battery, n_bands=10): """Construct 10-band charge bid from quantile
forecast. Charge bids use the lower tail of the price distribution. The battery wants to charge when prices are low.
""" power = battery["power_mw"] # Charge bands: price from high to low (NEM convention) # Place most capacity at low
price quantiles band_quantiles = [0.90, 0.80, 0.70, 0.60, 0.50, 0.40, 0.30, 0.20, 0.10, 0.05] bands = [] for i, q in
enumerate(band_quantiles): price = quantile_forecast[q] if 0.20 <= q <= 0.50: qty = power * 0.15 elif q <= 0.10: qty =
power * 0.05 else: qty = power * 0.10 bands.append({"band": i + 1, "price": price, "quantity": qty}) df =
pd.DataFrame(bands) df["quantity"] = df["quantity"] * power / df["quantity"].sum() return df
```

The Stochastic Dispatch Target

The scenario MPC from Chapter 9 produces a dispatch target -- the optimal charge or discharge for each interval, optimised across multiple price scenarios. The offer bands need to implement this target approximately, dispatching the right quantity at the right clearing prices.

The connection between the LP dispatch target and the offer bands is:

1. **Solve the scenario MPC** across N copula scenarios to get the optimal dispatch target d_t^* for each interval t
2. **Construct offer bands** that deliver approximately d_t^* when the clearing price equals the median forecast, and adjust the dispatch sensibly when the clearing price differs

The offer bands act as a **contingency plan**: if the price is higher than expected, the battery discharges more (the additional bands are dispatched). If the price is lower than expected, the battery discharges less (fewer bands are dispatched). The MPC's dispatch target is the "central case," and the offer bands spread the dispatch around it to handle forecast uncertainty.

Figure 12.3 — Translating a stochastic dispatch target into offer bands. The left panel shows the optimal dispatch from scenario MPC (Chapter 9) across 50 copula scenarios (Chapter 8). The middle panel shows the quantile forecast for the 6 PM interval. The right panel shows the resulting 10-band discharge offer, with band quantities concentrated around the median forecast price and smaller quantities in the tails. The shaded area shows the range of dispatch outcomes across different clearing prices.

The offer bands encode a conditional strategy. Unlike the LP, which produces a single dispatch quantity per interval, the offer bands produce a dispatch quantity that depends on the realised clearing price. This is strictly more powerful than the LP approach -- the battery's dispatch automatically adapts to the actual market conditions without needing to rebid. The LP asks "what is the best quantity to trade at the expected price?" The offer bands ask "what is the best quantity to trade at every possible price?"

Worked Example: One Day's Offer Construction

Let us walk through a complete example for the Mannum BESS on a summer day in SA1.

Setup: - Date: 15 January 2026 - Battery: 100 MW / 200 MWh, RTE = 87%, initial SoC = 50% - Forecast: quantile regression from Chapter 7, calibrated per Chapter 8, scenarios generated by Gaussian copula (Insert C, Chapter 8) - Dispatch strategy: chance-constrained MPC from Chapter 9, quantile pair (0.10, 0.90)

Step 1: Generate the probabilistic forecast.

The forecast for the 24-hour horizon (48 half-hour intervals) provides quantile predictions at levels 0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95. Key intervals shown below (04:00 = overnight, 10:00 = morning solar, 14:00 = peak solar, 17:30 = ramp hour, 19:00 = evening peak, 23:00 = late evening):

Period	q05	q25	q50	q75	q95
04:00	-\$5	\$15	\$28	\$42	\$65
10:00	-\$20	\$5	\$22	\$55	\$110
14:00	-\$50	-\$10	\$12	\$45	\$130
17:30	\$60	\$110	\$180	\$320	\$800
19:00	\$80	\$150	\$260	\$500	\$2,500

23:00	\$10	\$30	\$48	\$72	\$120
-------	------	------	------	------	-------

Step 2: Solve the chance-constrained MPC.

Using the $q_{0_1_0}$ / $q_{0_9_0}$ quantile pair, the LP produces the following dispatch schedule (selected intervals):

Period	MW	Action
04:00	-60	Charge
10:00	-80	Charge
14:00	-100	Max charge
17:30	+80	Discharge
19:00	+100	Max discharge
23:00	0	Idle

Step 3: Construct offer bands for the 19:00 interval.

The LP says "discharge 100 MW." The quantile forecast is: $q_{0_0_5} = \$80$, $q_{0_1_0} = \$110$, $q_{0_2_5} = \$150$, $q_{0_5_0} = \$260$, $q_{0_7_5} = \$500$, $q_{0_9_0} = \$1,200$, $q_{0_9_5} = \$2,500$.

The discharge offer bands:

Band	Price (\$/MWh)	Quantity (MW)	Logic
1	\$80	10	5th percentile -- dispatch floor
2	\$100	10	Between 5th and 10th percentile
3	\$110	10	10th percentile -- LP's conservative price
4	\$130	15	Between 10th and 25th percentile
5	\$150	15	25th percentile
6	\$200	15	Between 25th and 50th percentile
7	\$260	10	50th percentile -- median
8	\$500	8	75th percentile -- held for spikes
9	\$1,200	5	90th percentile -- held for significant spikes
10	\$2,500	2	95th percentile -- extreme events

If the actual price clears at \$180/MWh: bands 1-5 dispatch fully (60 MW) and band 6 dispatches partially (the remaining demand up to the clearing price). Total dispatch is approximately 65 MW at \$180/MWh.

If the actual price clears at \$600/MWh: bands 1-8 dispatch fully (93 MW). Revenue is much higher.

If the actual price clears at \$70/MWh: no bands dispatch (all bands priced above \$70). The battery holds its charge for later -- the forecast was wrong about this interval, and the offer bands protected against the loss.

Step 4: Settle and compare.

Mode	Clearing price	Dispatch (MW)	Revenue (half-hour)
Price-taker (historical)	\$245	100	\$12,250
Price-taker (offer bands)	\$245	85 (bands 1-6 + partial 7)	\$10,413
Price-maker (nempy)	\$228	82	\$9,348

The price-taker mode with direct LP dispatch overstates revenue by assuming the full 100 MW dispatches at the historical price. The offer bands naturally throttle dispatch when the price is below some band thresholds. The price-maker mode shows the additional revenue reduction from market impact -- the battery's 82 MW of discharge pushed the clearing price down by \$17/MWh from the historical \$245.

The revenue hierarchy: Price-taker LP > Price-taker with offer bands > Price-maker with offer bands. The first gap reflects the constraint of working within the 10-band offer structure (a coarser control than a continuous LP). The second gap reflects market impact. For the Mannum BESS, the combined gap is typically 8-15% of price-taker LP revenue.

The Full Price-Maker Backtest

Rolling-Origin Simulation

The price-maker backtest follows the same rolling-origin structure as Chapter 9 (and the shared backtest harness from Chapter 5), but replaces the simple price settlement with a nempy re-clearing at each interval:


```
def backtest_price_maker(forecasts, battery, region="SA1"): """Rolling-origin backtest with nempy price-maker simulation. For each forecast origin: 1. Solve chance-constrained MPC to get dispatch targets 2. Construct 10-band offers from targets + quantile forecast 3. For each interval in the execution window: a. Load historical bid stack from NEMOSIS b. Insert battery offers into nempy c. Clear the market d. Record dispatch and revenue at the new clearing price 4. Update battery state of charge 5. Roll forward to next origin Args: forecasts: Quantile forecasts for each origin. battery: Battery parameters dict. region: NEM region code. Returns: DataFrame with dispatch, clearing prices (original and new), revenue, and market impact. """ results = [] soc = battery["duration_h"] * battery["power_mw"] * 0.5 # Start at 50% for origin in forecast_origins: # Step 1: Solve MPC targets = solve_dispatch_lp( forecasts[origin], battery, method="chance_constrained", initial_soc=soc, ) # Step 2-3: Execute each interval for t in execution_window(origin): # Construct offers from target + quantile forecast if targets[t] > 0: # Discharge offer = construct_discharge_offer( forecasts[origin].quantiles[t], battery ) bid = empty_charge_bid(battery) else: # Charge offer = empty_discharge_offer(battery) bid = construct_charge_bid( forecasts[origin].quantiles[t], battery ) # nempy re-clearing result = simulate_price_maker(t, offer, bid) # Record results.append({ "interval": t, "original_price": actual_prices[t], "new_price": result["clearing_price"], "dispatch_mw": result["dispatch_mw"], "revenue": result["total_revenue"], "market_impact": actual_prices[t] - result["clearing_price"], }) # Update SoC soc = update_soc(soc, result["dispatch_mw"], battery) return pd.DataFrame(results)
```

Computational Considerations

The price-maker backtest is substantially more expensive than the price-taker backtest:

Component	Price-taker	Price-maker
Forecast generation	Same	Same
LP solve	~10 ms per origin	~10 ms per origin
Market clearing	Not needed	~0.5-2 s per interval (nempy)
Bid stack loading	Not needed	~0.1 s per interval (NEMOSIS)
Total for 1 year (half-hourly)	~5 minutes	~8-12 hours

The nempy clearing is the bottleneck. Each dispatch interval requires loading the complete bid stack (all generators' offers), building the LP, solving it, and extracting results. For a year of half-hourly data (17,520 intervals), this can take 8-12 hours on a modern laptop.

Strategies to manage this:

1. **Subsample intervals:** run the price-maker simulation on a representative subset (e.g., every 6th interval, or only peak hours) and interpolate
2. **Cache bid stacks:** download all bid stacks once and load from disk
3. **Parallelise:** each interval's clearing is independent; use multiprocessing
4. **Selective mode:** run price-taker for most intervals, switch to price-maker only during high-impact periods (high demand, steep supply stack)

Example -- selective price-maker simulation: Analyse a year of SA1 data and flag intervals where the market impact factor exceeds \$0.05/MW (roughly the threshold where 100 MW shifts the price by more than \$5). Typically, about 30% of intervals exceed this threshold. Run price-maker simulation only on these intervals and use price-taker settlement for the rest. This reduces computation time from 12 hours to about 4 hours while capturing >90% of the market impact effect.

Revenue Assessment and Benchmarks

Capture Ratio in Price-Maker Mode

The capture ratio framework from Chapter 9 extends naturally to price-maker mode. The key difference is that the perfect-foresight benchmark must also account for market impact:

```
CR_price_taker = Revenue_price_taker / Revenue_perfect_foresight_price_taker
CR_price_maker = Revenue_price_maker / Revenue_perfect_foresight_price_maker
```

The price-maker perfect foresight is computed by solving the dispatch LP with actual prices, then re-clearing each interval through nempy with the optimal dispatch as offers. The perfect-foresight revenue is lower in price-maker mode because even the perfect-foresight battery suffers market impact.

Metric	Price-taker	Price-maker	Difference
Perfect foresight revenue (SA1, annual)	\$12M	\$10.5M	-12.5%
Capture ratio (good forecast)	0.65-0.75	0.60-0.72	-3 to -5 pp
Annual revenue (good forecast)	\$7.8M-\$9.0M	\$6.3M-\$7.6M	-\$1.2M-\$1.7M

Benchmark targets for the Mannum BESS: The 0.50 capture ratio bar represents the minimum acceptable performance -- below this, the dispatch strategy is leaving too much money on the table. The target range is 0.65-0.75 in price-taker mode, corresponding to 0.60-0.72 in price-maker mode after accounting for market impact. These benchmarks should be reported against AEMO pre-dispatch forecasts as the baseline -- a strategy that merely follows AEMO's published pre-dispatch prices. A capture ratio that fails to beat the AEMO pre-dispatch baseline has no value; the operator could achieve the same result without any proprietary forecasting.

Revenue Decomposition by Source

A detailed revenue decomposition helps identify where the dispatch strategy gains and loses value:

Component	Taker \$M	Maker \$M
Base arbitrage (daily)	5.5	4.8
Spike capture (>\$300)	2.5	2.0
Negative price capture	0.8	0.9
Intra-day secondary	1.2	0.9
Total arbitrage	10.0	8.6
Market impact cost	--	-1.4

The price-maker total is 14% below the price-taker result due to self-cannibalization. Negative price capture is slightly higher for the price-maker because charging at negative prices increases demand, making the negative price less negative. Smaller intra-day spreads are more affected by market impact than the primary cycle.

Example -- monthly capture ratio comparison: Running the Mannum BESS backtest for July 2025 (winter, moderate volatility) and January 2026 (summer, high volatility): **July 2025:** Perfect foresight revenue = \$650K. Price-taker CR = 0.71 (\$462K). Price-maker CR = 0.67 (\$435K). Market impact cost = \$27K (4.2% of price-taker revenue). **January 2026:** Perfect foresight revenue = \$1,400K. Price-taker CR = 0.68 (\$952K). Price-maker CR = 0.62 (\$868K). Market impact cost = \$84K (8.8% of price-taker revenue). Market impact is proportionally larger in January because summer spikes occur on a steeper part of the supply stack, and the battery's discharge during these spikes has a larger price-depressing effect.

Strategy Search (Advanced)

The Methods Placement Principle

Before exploring advanced strategy search methods, it is worth restating where each optimisation approach fits in the broader curriculum:

The methods placement principle: Convex and stochastic optimisation -- the LP, scenario MPC, and chance-constrained MPC from Chapters 5 and 9 -- is the **core** of production battery dispatch. It is interpretable (the LP's dual variables tell you exactly why each decision was made), constraint-respecting (battery limits are enforced by construction), and fast enough for real-time MPC. Decision-focused learning (Chapter 16) wraps this convex core to train the forecaster end-to-end: the forecast parameters are adjusted to maximise dispatch revenue, not forecast accuracy. Reinforcement learning (RL) and genetic search only make sense **against the price-maker emulator**, where the optimal bid depends on other participants' bids and the bid-to-price mapping is non-convex. They come last in the curriculum -- after the convex baseline and emulator exist -- because without a solid baseline and a realistic simulator, there is nothing meaningful to improve upon and no way to evaluate whether the improvement is genuine.

The reason for this ordering is not merely pedagogical -- it reflects engineering reality. Production battery trading desks overwhelmingly use convex optimisation because:

1. **Interpretability:** Regulators and risk managers can audit the dispatch logic. An LP's dual variables explain every decision.
2. **Constraint satisfaction:** Battery limits (SoC bounds, ramp rates, cycle limits) are hard constraints in the LP. RL and genetic methods satisfy constraints only approximately, requiring penalty terms that must be tuned.
3. **Convergence guarantees:** The LP finds the global optimum in polynomial time. RL may not converge; genetic algorithms may settle on local optima.
4. **Speed:** The LP solves in milliseconds. RL requires thousands of episodes to train.

The one domain where convex optimisation falls short is **strategic bidding** against a price-maker emulator. When the battery's offers affect the clearing price, and the clearing price depends on other generators' offers (which are themselves strategic), the bid-to-revenue mapping is not convex. The optimal offer depends on the full bid stack -- a high-dimensional, non-smooth function that the LP cannot capture.

Reinforcement Learning for Bidding

In principle, reinforcement learning (RL) can learn to bid strategically in the price-maker emulator. The formulation:

- **State:** battery SoC, current time, recent prices, forecast quantiles, recent bid stack shape
- **Action:** the 10 band quantities for the current interval (continuous action space)
- **Reward:** revenue from the nempy clearing at each interval

- **Environment:** the price-maker emulator (nempy with historical bid stacks)

The RL agent observes the state, selects offer bands, nempy clears the market, the agent receives the revenue as reward, and the process repeats. Over thousands of training episodes, the agent learns which offer band configurations produce the most revenue in different market conditions.

Reinforcement learning (RL): A machine learning paradigm where an agent learns to make sequential decisions by interacting with an environment and receiving rewards. The agent's goal is to learn a policy -- a mapping from states to actions -- that maximises cumulative reward over time. For battery bidding, the environment is the price-maker emulator, the actions are the offer bands, and the rewards are dispatch revenue. RL excels when the optimal action depends on a complex, non-differentiable mapping from action to outcome -- exactly the situation with strategic bidding.

Challenges specific to NEM battery bidding with RL:

1. **High-dimensional action space:** 10 band quantities, each continuous, for both discharge and charge = 20 continuous actions per interval. This requires sophisticated RL algorithms (e.g., SAC, PPO with continuous actions).
2. **Slow environment:** each nempy clearing takes 0.5-2 seconds. Training for 10,000 episodes of 48 intervals each requires 250,000-1,000,000 clearings = days of computation.
3. **Non-stationarity:** the bid stack changes daily as generators enter, exit, and change their bidding behaviour. An RL policy trained on 2024 data may not generalise to 2025.
4. **Constraint enforcement:** the RL policy must respect battery constraints (SoC bounds, ramp rates). This requires either action masking or penalty terms.
5. **Sample efficiency:** historical data is limited (the NEM has run since 1998, but bid stack data at 5-minute resolution is available only from ~2009). RL may overfit to the training period.

Genetic Search for Offer Strategies

An alternative to RL is **genetic search** (or evolutionary optimisation), which searches the space of bidding strategies directly:

Genetic search (evolutionary optimisation): An optimisation method inspired by biological evolution. A population of candidate solutions (bidding strategies) is evaluated against a fitness function (revenue in the price-maker emulator). The best-performing strategies are selected, combined (crossover), and randomly modified (mutation) to create the next generation. Over many generations, the population evolves toward higher-fitness strategies. Genetic search is well-suited to problems with non-smooth, non-convex objective functions and discrete or mixed action spaces.

A practical setup:

1. **Parameterise the bidding strategy** as a small set of numbers: e.g., "discharge bands are placed at the $[a_1, a_2, \dots, a_{10}]$ quantiles of the forecast, with equal quantities." This reduces the search space from 20 continuous variables per interval to a handful of strategy parameters.
2. **Evaluate each strategy** by running the full price-maker backtest (or a subsample) and computing the annual revenue.
3. **Evolve the population** using standard genetic operators: tournament selection, arithmetic crossover, Gaussian mutation.

```
# Genetic search for bidding strategy (simplified)
import numpy as np
def parameterised_offer(quantile_forecast, strategy_params, battery):
    """Construct offers using a parameterised strategy.
    strategy_params: array of 10 quantile levels for discharge bands.
    e.g. [0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95]
    """
    power = battery["power_mw"]
    bands = []
    for i, q in enumerate(sorted(strategy_params)):
        price = np.interp(q, list(quantile_forecast.keys()), list(quantile_forecast.values()))
        qty = power / len(strategy_params)
        # Equal allocation
        bands.append({"band": i + 1, "price": price, "quantity": qty})
    return pd.DataFrame(bands)

def fitness(strategy_params, forecasts, battery, sample_days):
    """Evaluate a strategy by running price-maker backtest on sample days.
    """
    total_revenue = 0
    for day in sample_days:
        for t in day_intervals(day):
            offer = parameterised_offer(forecasts[t], strategy_params, battery)
            result = simulate_price_maker(t, offer, empty_charge_bid(battery))
            total_revenue += result["total_revenue"]
    return total_revenue

def genetic_search(forecasts, battery, n_generations=50, pop_size=30):
    """Search for optimal bidding strategy using genetic algorithm.
    """
    # Initial population: random quantile level vectors
    population = [np.sort(np.random.uniform(0.01, 0.99, 10)) for _ in range(pop_size)]
    sample_days = select_representative_days(n=20)
    for gen in range(n_generations):
        # Evaluate fitness scores
        scores = [fitness(p, forecasts, battery, sample_days) for p in population]
        # Select top half
        sorted_idx = np.argsort(scores)[::-1]
        survivors = [population[i] for i in sorted_idx[:pop_size // 2]]
        # Crossover and mutation
        children = []
        for _ in range(pop_size // 2):
            p1, p2 = np.random.choice(survivors, 2, replace=False)
            alpha = np.random.uniform(0, 1)
            child = alpha * survivors[p1] + (1 - alpha) * survivors[p2]
            child += np.random.normal(0, 0.02, 10)
            child = np.clip(np.sort(child), 0.01, 0.99)
            children.append(child)
        population = survivors + children
        print(f"Gen {gen}: best revenue = ${max(scores):.0f}")
        best_idx = np.argmax(scores)
    return population[best_idx]
```

The genetic search typically finds strategies that outperform the convex baseline by 2-5% in the price-maker emulator. The improvement comes from learning to:

- **Shade offers** slightly above the competitive level in intervals where the supply stack is steep (capturing higher prices at the cost of slightly lower dispatch probability)
- **Split discharge** across multiple intervals rather than concentrating in the single highest-price interval (reducing market impact)
- **Time charge** to avoid demand peaks in the overnight market (reducing the price-inflating effect of charging)

When to use strategy search: Start with the convex baseline (Chapter 9). If the price-maker backtest shows market impact costs exceeding 5% of revenue, strategy search is worth investigating. For the Mannum BESS in SA1, market impact typically costs 5-10% of revenue, placing it squarely in the range where strategy search can add value. For a smaller battery (30 MW) in a larger region (NSW1), market impact is negligible and strategy search offers no benefit over the convex baseline.

Putting It Together: The Complete Bidding Pipeline

The complete pipeline from forecast to market participation chains together all the components developed across the course:

1. **Data ingestion** (Chapter 1): pull NEMOSIS price and demand data, NEMSEER forecasts, ERA5 weather data
2. **Feature engineering** (Chapters 3-4): compute net load, calendar features, weather lags, interconnector flows
3. **Point forecasting** (Chapters 6-7): LEAR and GBT models for the price median
4. **Probabilistic forecasting** (Chapter 8): quantile regression, QRA combination, copula scenario generation
5. **Dispatch optimisation** (Chapter 9): scenario MPC or chance-constrained MPC to get dispatch targets
6. **Offer construction** (this chapter): translate dispatch targets and quantile forecasts into 10-band offers
7. **Market simulation** (this chapter): clear offers through nempy in price-maker mode
8. **Settlement** (this chapter): compute revenue at the new clearing price
9. **Performance evaluation**: capture ratio, AEMO pre-dispatch comparison, market impact decomposition

Each step builds on the previous ones. The forecast quality from Chapters 6-8 determines how well the offer bands are placed. The dispatch strategy from Chapter 9 determines the central dispatch target. The offer construction from this chapter determines how the target is mapped into the NEM's 10-band format. And the price-maker simulation from this chapter determines the actual revenue after market impact.

Example -- end-to-end revenue comparison for the Mannum BESS: | Stage | Annual revenue | Capture ratio | Notes |
|-----|-----|-----|-----| | Perfect foresight (price-taker) | \$12.0M | 1.00 | Theoretical maximum | | Perfect foresight (price-maker) | \$10.5M | -- | Market impact even with perfect knowledge | | Chance-constrained MPC (price-taker) | \$8.4M | 0.70 | Chapter 9 result | | Chance-constrained MPC (price-maker) | \$7.2M | 0.69 (of PM PF) | This chapter -- realistic assessment | | AEMO pre-dispatch baseline (price-taker) | \$6.0M | 0.50 | Benchmark to beat | | Naive similar-day baseline (price-taker) | \$4.8M | 0.40 | Chapter 5 baseline | The gap between "chance-constrained MPC (price-taker)" and "chance-constrained MPC (price-maker)" is \$1.2M/year -- the cost of market impact for the Mannum BESS. The gap between the price-maker result and the AEMO pre-dispatch baseline is \$1.2M/year -- the value added by proprietary forecasting and optimisation. Both gaps are large enough to justify the investment in the full pipeline.

FCAS Co-Optimisation in Bidding

Energy and FCAS Offers

The NEM co-optimises energy and FCAS (Frequency Control Ancillary Services -- introduced in Chapter 11) simultaneously. This means the battery's energy offers and FCAS offers interact: capacity allocated to FCAS enablement cannot simultaneously be dispatched for energy, and vice versa.

For the Mannum BESS, the offer structure includes both energy and FCAS bands:

- **Energy discharge:** 10 price-quantity bands (as described above)
- **Energy charge:** 10 price-quantity bands
- **FCAS raise:** up to 10 price-quantity bands for each of the four raise services (contingency 6s, 60s, 5min; regulation raise)
- **FCAS lower:** up to 10 price-quantity bands for each of the four lower services

NEMDE co-optimises all of these simultaneously, allocating the battery's capacity to whichever combination of energy and FCAS maximises total market surplus.

Example -- energy-FCAS tradeoff: The Mannum BESS has 100 MW of capacity. If it offers 80 MW for energy discharge and 20 MW for FCAS contingency raise, NEMDE might dispatch: - 70 MW of energy discharge at \$130/MWh (energy revenue = $\$130 \cdot 70 / 12 = \$758/\text{interval}$) - 20 MW of FCAS raise enablement at \$15/MW/h (FCAS revenue = $\$15 \cdot 20 / 12 = \$25/\text{interval}$) - 10 MW unallocated (the energy dispatch target was below the full 80 MW offered) The total revenue is \$783/interval. If the battery had offered all 100 MW for energy, it might have dispatched 90 MW at \$125/MWh (lower price due to more supply) = \$938/interval. In this case, the energy-only strategy wins. But during periods of high FCAS prices (which can reach \$300/MW/h or more), the FCAS allocation is far more profitable. The dispatch LP from Chapter 9 can be extended to co-optimize energy and FCAS dispatch (covered in Chapter 11). The offer construction here must then produce consistent energy and FCAS bands.

Loss Factors and Regional Considerations

In the NEM, generators do not receive the regional reference price directly -- they receive the price adjusted by a **marginal loss factor (MLF)** that accounts for transmission losses between the generator's connection point and the regional reference node.

Marginal loss factor (MLF): A multiplier applied to a generator's revenue to account for transmission losses. An MLF of 0.98 means the generator receives 98% of the regional reference price -- 2% is lost in transmission. MLFs are published annually by AEMO and depend on the generator's location in the network. Generators far from the regional reference node or on congested network paths tend to have lower MLFs.

The Mannum BESS, located in regional South Australia, has an MLF that depends on its network position. In price-maker mode, the MLF affects both the battery's received price and its market impact:

- **Revenue** = $\text{dispatch_mw} \cdot \text{clearing_price} \cdot \text{MLF} / 12$ (for a 5-minute interval)
- **Market impact** is computed at the regional reference node, but the battery's effective impact on its own revenue is scaled by MLF

For offer construction, the band prices should be adjusted by the MLF: if the MLF is 0.97, a band priced at \$100/MWh receives only \$97/MWh at settlement. The offer should account for this by pricing bands slightly higher (dividing by MLF).

Exercises

Exercise 12.1: Offer Aggressiveness and the Revenue-Risk Tradeoff

Problem: The Mannum BESS faces a forecast for the 6 PM interval: $q_{0.1_0} = \$80$, $q_{0.5_0} = \$150$, $q_{0.9_0} = \$400$. You have 100 MW of discharge capacity. Consider three offer strategies:

- (a) **Aggressive:** Place all 100 MW at \$80/MWh (the 10th percentile). You will be dispatched whenever the price exceeds \$80.
- (b) **Moderate:** Spread capacity evenly across five bands from \$80 to \$400 (20 MW at \$80, \$120, \$180, \$260, \$400).
- (c) **Conservative:** Place all 100 MW at \$400/MWh (the 90th percentile). You will only be dispatched during spikes.

For each strategy, compute the expected revenue across 1,000 simulated prices drawn from the forecast distribution (assume log-normal with the given quantiles). Also compute the 10th percentile revenue (the "downside case") and the probability of zero dispatch.

Worked solution

Step 1: Fit a log-normal distribution to the quantiles.

$q_{0.1} = \exp(\mu - 1.28 \cdot \sigma) = \80 $q_{0.5} = \exp(\mu) = \$150$ $q_{0.9} = \exp(\mu + 1.28 \cdot \sigma) = \400 From $q_{0.5}$: $\mu = \ln(150) = 5.01$ From $q_{0.1}$: $5.01 - 1.28 \cdot \sigma = \ln(80) = 4.38$ $\sigma = (5.01 - 4.38) / 1.28 = 0.49$ Check $q_{0.9}$: $\exp(5.01 + 1.28 \cdot 0.49) = \exp(5.64) = \281

The check does not exactly match \$400 — the real distribution is right-skewed beyond log-normal. Accept the approximation for this exercise.

Step 2: Define dispatch rules for each strategy.

Draw 1,000 prices: $p \sim \text{LogNormal}(5.01, 0.49^2)$.

- (a) **Aggressive:** dispatch = 100 MW if $p > \$80$, else 0. Revenue = $100 \cdot p / 2$.
- (b) **Moderate:** dispatch = 20 · (number of bands where $p \geq \text{band price}$). At $p = \$160$, bands at \$80 and \$120 clear → 40 MW dispatched. Revenue = $40 \cdot \$160 / 2 = \$3,200$.
- (c) **Conservative:** dispatch = 100 MW if $p \geq \$400$, else 0.

Step 3: Compare simulation results.

Strategy	E[Revenue] (\$/half-hr)	q _{0.1} Revenue	P(zero dispatch)
Aggressive	\$8,750	\$4,200	10%
Moderate	\$6,200	\$1,600	10%
Conservative	\$2,100	\$0	73%

Step 4: Interpret.

- **Aggressive** has the highest expected revenue but discharges at every price above \$80, possibly below the round-trip cost of charging. It exhausts the battery early, leaving nothing for later spikes.
- **Moderate** preserves some capacity for high prices by dispatching partial quantities at moderate prices.
- **Conservative** earns the most per MWh dispatched but misses most opportunities — profitable only if spikes are frequent and severe.

The **tradeoff** depends on state of charge (if nearly full, be aggressive), the remaining forecast horizon (if more high-price intervals expected, be conservative), and forecast confidence (if the 10th-90th range is wide, spread the offers more).

Exercise 12.2: Finding the Price-Taker Threshold

Problem: Run the price-maker simulation for hypothetical batteries of sizes 10, 25, 50, 100, 150, 200, and 300 MW (all 2-hour duration, 87% RTE) in SA1. For each size, compute:

- (a) The average market impact (\$/MWh price change per interval when the battery is dispatching)
- (b) The annual revenue loss from market impact (price-maker revenue minus price-taker revenue)
- (c) The revenue loss as a percentage of price-taker revenue

Plot (c) against battery size. At what size does the price-taker assumption start to cost real money (defined as >3% revenue loss)?

Worked solution

Step 1: Run price-maker backtest for each battery size.

Using one year of SA1 data (2025), chance-constrained MPC (quantile pair 0.10/0.90):

Battery size (MW)	Avg impact (\$/MWh)	Revenue loss (\$/yr)	Loss (% of PT)
10	\$0.4	\$18K	0.5%
25	\$1.1	\$95K	1.2%
50	\$2.5	\$340K	2.8%
100	\$5.8	\$1,200K	6.2%
150	\$9.5	\$2,800K	10.1%
200	\$14.2	\$5,100K	14.8%
300	\$24.0	\$11,500K	23.5%

Step 2: Find the 3% threshold.

The 3% threshold is crossed at approximately **55 MW** in SA1. Below 55 MW, price-taker is adequate. Above 55 MW, price-maker modelling is necessary.

Step 3: Interpret the scaling.

Doubling the battery from 100 to 200 MW roughly quadruples the revenue loss (\$1.2M → \$5.1M), while only doubling price-taker revenue (\$19.4M → \$34.5M). This quadratic scaling means market impact grows much faster than capacity.

Key observations:

- **Region matters.** SA1 (800–2,500 MW demand) has a low threshold. NSW1 (5,000–12,000 MW) would be ~200–300 MW.
- **Time of day matters.** Overnight low demand (~800 MW): even 25 MW has measurable impact. Summer afternoon peaks (~2,500 MW): 100 MW has minimal impact.
- **Crowding compounds.** Three 100 MW batteries in SA1 create far worse combined impact than any one alone — this is the **crowding problem** in battery investment.

Exercise 12.3 (Optional Advanced): Strategy Search vs Convex Baseline

Problem: Implement a simple genetic search that optimises the quantile levels used for discharge offer band placement. Use the parameterised strategy from the "Genetic Search" section above. Run the search against 20 representative days in SA1 using the price-maker emulator.

(a) What quantile levels does the genetic search converge to? How do they differ from the uniform spacing [0.05, 0.15, ..., 0.95]?

(b) What is the revenue improvement over the convex baseline (chance-constrained MPC with uniform band spacing)?

(c) Does the optimised strategy generalise to an out-of-sample test period (10 days not in the training set)?

Worked solution

Running the genetic search with population 30, 50 generations, on 20 SA1 days from 2025.

(a) Converged quantile levels:

Band	Uniform spacing	Genetic search result
1	0.05	0.08
2	0.15	0.14
3	0.25	0.22
4	0.35	0.33
5	0.45	0.42
6	0.55	0.55
7	0.65	0.68
8	0.75	0.78
9	0.85	0.88
10	0.95	0.97

The search pushes top bands toward the tails and compresses lower bands — reserving more capacity for high-price outcomes. The shift is subtle; the improvement comes from fine-tuning to the SA1 supply stack shape.

(b) Revenue improvement:

Strategy	Revenue (20 training days)	Improvement
----------	----------------------------	-------------

Convex baseline (uniform bands)	\$185,000	-
Genetic search (optimised bands)	\$193,500	+4.6%

Two effects: (1) higher top bands (0.88, 0.97 vs 0.85, 0.95) retain more capacity for spikes, capturing \$4K–\$6K extra on days above \$500/MWh; (2) compressed lower bands reduce market impact at moderate prices.

(c) Out-of-sample performance:

Strategy	Revenue (10 test days)	Improvement
Convex baseline (uniform bands)	\$98,000	-
Genetic search (optimised bands)	\$101,000	+3.1%

The improvement generalises but is smaller (3.1% vs 4.6%) — the search over-fits slightly to training bid stack patterns. Still significant at $p < 0.05$ (paired t-test).

Practical guidance: 3–5% improvement $\approx$ \$200K–\$400K/year for Mannum. Economically meaningful but requires maintaining a price-maker simulation pipeline. The convex baseline remains the production workhorse; strategy search is a refinement for large batteries in small regions.

Glossary

Term	Definition
Offer band	A price-quantity pair in a generator's offer; the NEM allows up to 10 bands per unit
Rebidding	Updating the quantity allocation across offer bands before gate closure
Gate closure	The deadline for submitting offer updates (~5 minutes before the dispatch interval in the NEM)
Price-taker	A market participant too small to affect the clearing price
Price-maker	A market participant large enough to move the clearing price through its own actions
Market impact	The change in clearing price caused by a generator's dispatch
Residual demand	The demand remaining after all other generators' supply is accounted for; the demand curve faced by a single generator
Integrated resource	A resource that can both generate and consume (e.g., a battery), participating on both sides of the energy market
Bid stack	The collection of all generators' offer bands, stacked in ascending price order
Market price cap (MPC)	The maximum allowed price in the NEM (\$16,600/MWh as of 2024-25)
Market floor price (MFP)	The minimum allowed price in the NEM (-\$1,000/MWh)
Uniform pricing	All dispatched generators receive the same clearing price, regardless of offer price
NEMDE	NEM Dispatch Engine -- AEMO's algorithm for clearing the energy and FCAS markets
nempy	Open-source Python implementation of NEMDE by UNSW-CEEM
Marginal loss factor (MLF)	Multiplier adjusting a generator's revenue for transmission losses
Self-cannibalization	The effect where a battery's own dispatch reduces the price it receives
FCAS co-optimisation	Simultaneous clearing of energy and FCAS markets, allocating capacity across services
Capture ratio (CR)	Actual dispatch revenue divided by perfect-foresight revenue
Genetic search	Evolutionary optimisation method for finding high-performing bidding strategies

Summary

This chapter bridges the gap between the mathematical dispatch optimum (the LP from Chapter 9) and the physical NEM market mechanism. A battery does not choose its dispatch quantity directly -- it submits price-quantity offer bands and is dispatched by NEMDE based on where those bands sit in the aggregate supply stack. The 10-band offer structure is both a constraint (coarser than the continuous LP) and an advantage (the dispatch automatically adapts to the realised price without needing to rebid).

The chapter introduced two simulation modes. **Price-taker mode** -- which the course used through Chapter 10 -- assumes the battery is too small to affect the clearing price. This is a good approximation for batteries below about 5% of regional minimum demand. **Price-maker mode**

uses nempy to re-clear the market with the battery's offers inserted into the historical bid stack, revealing the market impact: the battery's own dispatch shifts the clearing price against it. For the 100 MW Mannum BESS in SA1, market impact costs roughly 5-10% of price-taker revenue, narrowing the arbitrage spread through self-cannibalization on both the charge and discharge sides.

Offer construction translates the probabilistic forecast (Chapter 8) and the MPC dispatch target (Chapter 9) into 10-band offers by mapping forecast quantiles to band prices. The bands encode a conditional strategy: the battery's dispatch depends on the realised clearing price, automatically adapting to forecast errors without rebidding. The offer aggressiveness tradeoff -- between expected revenue and dispatch risk -- is controlled by how the capacity is spread across the price range.

Advanced strategy search (RL or genetic optimisation) can squeeze an additional 3-5% revenue from the price-maker emulator by learning to shade offers strategically. But the convex LP baseline remains the production core: interpretable, constraint-respecting, and fast. Strategy search is a refinement for large batteries in small regions, not a replacement for the optimisation framework built in Chapters 5 and 9. The capture ratio target remains 0.65-0.75 in price-taker mode (0.60-0.72 in price-maker mode), benchmarked against AEMO pre-dispatch forecasts. Below the 0.50 bar, the dispatch strategy needs fundamental improvement before offer refinement adds value.